

ResNet-50: Image Classification API

AIE494 Project Report

1. รายละเอียดโมเดล

ResNet-50 (Residual Network-50) เป็นสถาปัตยกรรมโครงข่ายประสาทเทียมเชิงลึก (Deep Neural Network) ประเภท Convolutional Neural Network พัฒนาโดย He และคณะ (2016) ประกอบด้วยชั้นการเรียนรู้ทั้งหมด 50 ชั้น

จุดเด่นของโมเดลคือแนวคิด Residual Connection หรือ Skip Connection ซึ่งช่วยถ่ายโอนข้อมูลข้ามชั้นโดยตรง แก้ปัญหา Vanishing Gradient ที่พบในโครงข่ายลึก ทำให้โมเดลที่มีจำนวนชั้นสูงได้อย่างมีประสิทธิภาพ

ResNet-50 ถูกนำไปใช้งานหลักใน 5 ด้าน ได้แก่ การจำแนกประเภทภาพ (Image Classification), การตรวจจับวัตถุ (Object Detection) ในฐานะ Backbone Network ร่วมกับ Faster R-CNN หรือ YOLO, การถ่ายโอนการเรียนรู้ (Transfer Learning) บนชุดข้อมูล ImageNet, การแยกส่วนภาพ (Semantic Segmentation) และการสกัดคุณลักษณะ (Feature Extraction) สำหรับงาน Downstream อื่น ๆ

2. ผลการทำ Model Optimization

ทดสอบ 3 รูปแบบ ได้แก่ PyTorch (ต้นฉบับ), ONNX และ Quantized Int8 โดยเปรียบเทียบขนาดและความเร็วในการ Inference ดังนี้

ตารางที่ 1 ผลการเปรียบเทียบ Size และ Latency

โมเดล	ขนาด (MB)	Latency (ms)	ลดขนาด (%)	ลด Latency (%)
Original (PyTorch)	97.74	40.16	—	—
ONNX	97.43	9.25	0.32%	76.97%
Quantized	91.58	9.73	6.30%	75.77%

การแปลงเป็น ONNX ช่วยลด Latency ได้มากที่สุด (76.97%) โดยขนาดโมเดลแทบไม่เปลี่ยน ส่วน Quantized Int8 ลดขนาดโมเดลได้ดีกว่า (6.30%) และยังรักษาความเร็วได้ใกล้เคียงกัน ทั้งสองแบบเร็วกว่าโมเดลต้นฉบับมากกว่า 4 เท่า

3. Error Handling และ Data Validation

ระบบออกแบบ Validation เป็น 3 ชั้น (3-Layer Defense) แต่ละชั้นดักจับ Error คนละประเภท และตอบกลับด้วย HTTP Status Code ที่เหมาะสมเสมอ

ชั้นที่ 1 — Input Validation (API Layer)

ทุก Request ที่เข้ามาที่ POST /predict ผ่านการตรวจสอบ 3 ด้านตามลำดับ ก่อนส่งเข้า Model

- Content-Type check — ถ้าไม่ใช่ image/* → ส่งกลับ 400
- File size check — ถ้าเกิน 5MB → ส่งกลับ 400
- PIL integrity check — ถ้าไฟล์เสีย → ส่งกลับ 400 (เป็น ground truth เดียวที่เชื่อถือได้ เพราะ Content-Type สามารถปลอมได้จาก client)

ชั้นที่ 2 — Query Parameter Validation (Pydantic + Regex)

ค่า model ที่รับเข้ามาถูก validate ด้วย Pydantic Annotated และ regex pattern ก่อนที่ FastAPI จะ route เข้า handler

ค่าที่นอกเหนือจาก original, onnx, quantized จะได้รับ 422 พร้อม error detail อัตโนมัติโดยไม่ต้องเขียน logic เพิ่ม

```
ModelType = Annotated[str, Query(pattern="^(original|onnx|quantized)$")]
```

ชั้นที่ 3 — Output Schema Validation (Pydantic Response Model)

Response ทุกตัวถูกบังคับโครงสร้างผ่าน Pydantic model ทำให้ client ได้ข้อมูลครบเสมอ FastAPI ใช้

response_model=PredictionResponse เพื่อ serialize และ validate output ก่อน return ทุกครั้ง

HTTP Status Code ที่ระบบใช้

Status Code	กรณีที่เกิด
200 OK	สำเร็จทุกกรณี
400 Bad Request	ไฟล์ไม่ใช่รูปภาพ / ไฟล์เสีย / ไฟล์ใหญ่เกิน 5MB
404 Not Found	ขอรูปภาพที่ไม่มีใน storage (/images/{id})
422 Unprocessable Entity	model parameter ไม่ถูกต้อง (auto จาก Pydantic)

Status Code	กรณีที่เกิด
500 Internal Server Error	เกิดจาก unexpected error เท่านั้น (ไม่ใช่ input error)

การจัดการ Database Error (Graceful Degradation)

ระบบออกแบบให้ทำงานได้แม้ไม่มี Database (เช่น บน HF Spaces ที่ไม่มี PostgreSQL) โดยไม่ให้ผู้ใช้เห็น error — endpoint /predict ยังทำนายได้ปกติ และ /classifications คืน empty list แทน 500

สถานการณ์	พฤติกรรมของระบบ
DB ออนไลน์	บันทึกและดึงประวัติได้ปกติ
DB ออฟไลน์	ทำนายได้ปกติ, ประวัติเป็น []
DB ออฟไลน์ + /predict	ตอบกลับผลทำนายพร้อม UUID ที่ generate เอง

4. ผลการทดสอบด้วย JMeter

ทดสอบโหลดด้วย Apache JMeter (500 requests) พบอัตรา Error 7.80% และ Average Response Time ประมาณ 30,468 ms

โดยสาเหตุหลักมาจาก 3 ปัจจัย

- Model ช้า — Inference time สูงอยู่แล้ว เนื่องจากรันบน CPU
- ไม่มี Scaling — HF Free Tier เป็น CPU only ไม่มี Autoscale
- Queue / Blocking — Request ต่อคิวเมื่อโหลดสูง ทำให้ Latency พุ่งตามจำนวน Concurrent users

Source file	"result.jl"
Start Time	"5/4/26 8:05 PM"
End Time	"5/4/26 8:11 PM"
Filter for display	"=

Apdex	T (Toleration threshold)	F (Frustration threshold)	Label
0.000	500 ms	1 sec 500 ms	Total
0.000	500 ms	1 sec 500 ms	HTTP Request

A pie chart showing the results of the final exam. The chart is divided into two segments: a large green segment representing 'PASS' at 92.2%, and a small red segment representing 'FAIL' at 7.8%. A legend to the right of the chart identifies the colors: red for 'FAIL' and green for 'PASS'.

Result	Percentage
PASS	92.2%
FAIL	7.8%

Requests		Executions				Response Times (ms)							Throughput	Network (KB/sec)	
Label	#Samples	FAIL	Error %	Average	Min	Max	Median	90th pct	95th pct	99th pct	Transactions/s	Received	Sent		
Total	500	39	7.80%	30468.10	1584	65403	27529.50	60383.30	62217.15	63604.97	1.40	1.36	936.94		
HTTP Request	500	39	7.80%	30468.10	1584	65403	27529.50	60383.30	62217.15	63604.97	1.40	1.36	936.94		

Requests		Executions				Response Times (ms)					Throughput	Network (Kb/sec)		
Label	#Samples	FAIL	Error %	Average	Min	Max	Median	90th pct	95th pct	99th pct	Transactions/s	Received	Sent	
Total	500	39	7.80%	30468.10	1584	65403	27529.50	60383.30	62217.15	63604.97	1.40	1.36	936.94	
HTTP Request	500	39	7.80%	30468.10	1584	65403	27529.50	60383.30	62217.15	63604.97	1.40	1.36	936.94	

Type of error	Number of errors	% in errors	% in all samples
Non HTTP response code: java.net.SocketTimeoutException/Non HTTP response message: Read timed out	39	100.00%	7.80%

Sample	#Samples	#Errors	Error	#Errors	Error	#Errors	Error	#Errors	Error	#Errors	Error	#Errors
Total	500	39	Non HTTP response code: java.net.SocketTimeoutException/Non HTTP response message: Read timed out	39								
HTTP Request	500	39	Non HTTP response code: java.net.SocketTimeoutException/Non HTTP response message: Read timed out	39								

5. System Architecture และ CI/CD Pipeline

ภาพรวมระบบ

ระบบแบ่งออกเป็น 3 ชั้น (3-Tier Architecture) ได้แก่ Frontend, Backend API และ Database โดยมีระบบ AI Inference แยกเป็น Process pool เพื่อป้องกัน API blocking

- Frontend — React 18 + Vite รับอัปโหลดรูปภาพและแสดงผลการจำแนก
- Backend — FastAPI + Uvicorn รองรับ async I/O และ ProcessPoolExecutor สำหรับ CPU Inference
- Database — PostgreSQL + SQLAlchemy (async) บันทึกประวัติการจำแนก

CI/CD Pipeline (GitHub Actions)

เมื่อ push ขึ้น main branch GitHub Actions จะรัน 2 job ตามลำดับ

- Job: test — Setup Python 3.11, ติดตั้ง dependencies, Download + แปลง ResNet-50 เป็น ONNX และ Quantize เป็น Int8, สร้าง DB tables และรัน pytest
- Job: deploy — รันเฉพาะเมื่อผ่าน test ทั้งหมด โดย upload backend/ ไปยัง HF Space ซึ่งจะ trigger Docker build และรัน uvicorn:7860 อัตโนมัติ

สรุปเทคโนโลยีที่ใช้

Layer	Technology	หน้าที่
Frontend	React 18 + Vite	UI อัปโหลดและดูผลลัพธ์
Backend	FastAPI + Uvicorn	REST API, async I/O
Inference	ONNX Runtime + ProcessPoolExecutor	CPU inference ป้องกัน event loop block
Model	ResNet-50 (PyTorch → ONNX → Int8)	Image classification 1,000 classes
Database	PostgreSQL + SQLAlchemy (async)	บันทึกประวัติการจำแนก
Container	Docker + Docker Compose	Packaging และ local development
CI/CD	GitHub Actions	Auto test + deploy
Cloud	Hugging Face Spaces (Docker SDK)	Production hosting

อ้างอิง

He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 770–778.